PROJEKTDOKUMENTATION

Homeserver mit Pi-hole, Tailscale und Monitoring

Werbeblocker, Reverse Proxy, eigene Cloud und Monitoring auf ausgemusterter Hardware, erreichbar zuhause und unterwegs, ohne offenen Port ins Internet

Angabe	Wert
Autor	Tscheky
Ausbildung	IT Netzwerktechnik
Stand	23. Oktober 2026
Repository	github.com/Tscheky0/homeserver-pihole-tailscale

Inhaltsverzeichnis

1 Einleitung 4
1.1 Ziel des Projekts 4
1.2 Ausgangslage 4
1.3 Kennzahlen 4
2 Hardware und Netzwerk 5
3 Teil 1: Grundsystem

Inhalt

1 Einleitung 4
1.1 Ziel des Projekts 4
1.2 Ausgangslage 4
1.3 Kennzahlen 4
2 Hardware und Netzwerk 5
3 Teil 1: Grundsystem 6
3.1 Ubuntu Server und Netplan 6
3.2 Docker und Portainer 6
3.3 Pi-hole 6
3.4 Tailscale 6
3.5 SSH Härtung 6
4 Teil 2: Erweiterung 7
4.1 Was macht was 7
4.2 Reverse Proxy, Namen und HTTPS 8
4.3 Monitoring mit Prometheus und Grafana 8
4.4 Uptime Kuma und Alarme 8
4.5 Nextcloud 9
4.6 Homepage 9
4.7 Speicherbudget 9
5 Sicherheit 10
6 Fehlersuche: Probleme und Lösungen 11
6.1 Grundsystem 11
6.2 Erweiterung 11
7 Ergebnis 12
8 Was ich gelernt habe 14

..... 6
3.1 Ubuntu Server und Netplan 6
3.2 Docker und Portainer 6
3.3 Pi-hole 6

3.4 Tailscale 6
3.5 SSH Härtung 6
4 Teil 2: Erweiterung 7
4.1 Was macht was 7
4.2 Reverse Proxy, Namen und HTTPS 8
4.3 Monitoring mit Prometheus und Grafana 8
4.4 Uptime Kuma und Alarme 8
4.5 Nextcloud 9
4.6 Homepage 9
4.7 Speicherbudget 9
5 Sicherheit 10
6 Fehlersuche: Probleme und Lösungen 11
6.1 Grundsystem 11
6.2 Erweiterung 11
7 Ergebnis 12
8 Was ich gelernt habe 14

1 Einleitung

1.1 Ziel des Projekts

Aus einem ausgemusterten PC entsteht ein Homeserver, der im ganzen Haushalt und unterwegs Werbung und Tracking auf DNS Ebene blockt. Im zweiten Schritt wird er zu einem kleinen Rechenzentrum ausgebaut: mit Reverse Proxy und HTTPS, eigener Cloud, Monitoring mit Diagrammen, Ausfallalarmen aufs Handy und einer gemeinsamen Startseite.

Eine feste Vorgabe gilt für das ganze Projekt: **kein einziger Port wird ins Internet geöffnet.** Der Zugriff von außen läuft ausschließlich über das Mesh VPN Tailscale.

1.2 Ausgangslage

Der Server hat weder WLAN noch eine Kabelverbindung zum Router. Er hängt per LAN Kabel direkt an einem Windows PC, der seine WLAN Verbindung über die **Windows Internetverbindungsfreigabe (ICS)** weitergibt. Dieser Aufbau hat mehrere echte Netzwerkprobleme verursacht. Ihre Analyse und Lösung bilden den Kern des Projekts.

1.3 Kennzahlen

Kennzahl	Wert
Geblockte Domains	280.282 (StevenBlack und Hagezi Pro)
Offene Ports ins Internet	0
Container	15 in 7 Stacks
Überwachte Dienste	10 Monitore, Alarm per Push aufs Handy
Hardware	Alter PC, 4 Kerne, 4 GB RAM, 500 GB HDD
Fernzugriff	Tailscale, SSH nur mit Ed25519 Schlüssel

2 Hardware und Netzwerk

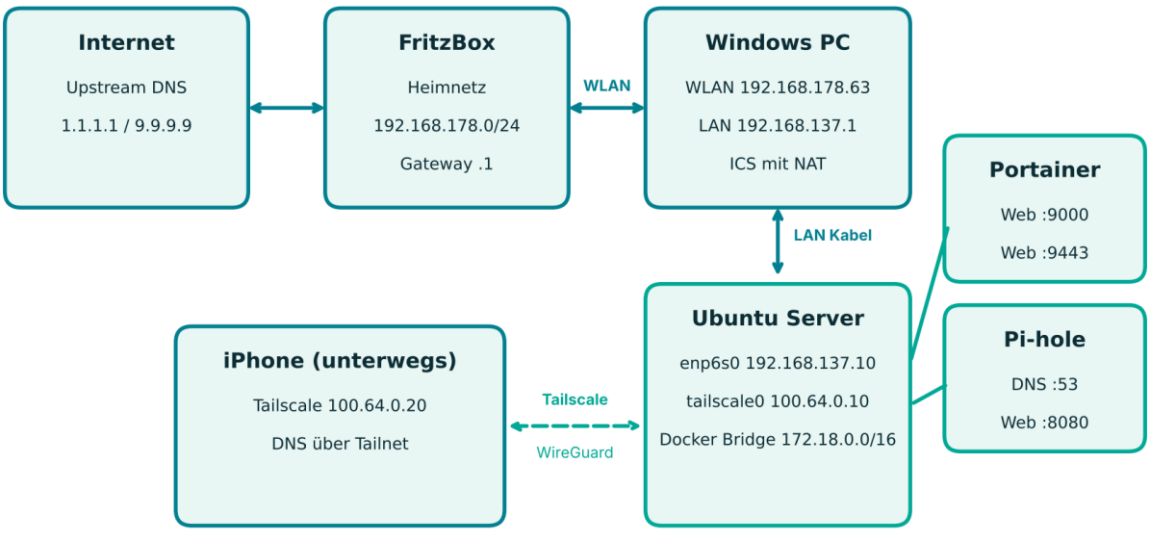


Abbildung 1: Netzwerkaufbau mit Windows ICS und Tailscale

Gerät oder Schnittstelle	Adresse	Rolle
FritzBox	`192.168.178.1`	Gateway im Heimnetz
Windows PC, WLAN	`192.168.178.x`	Uplink ins Heimnetz
Windows PC, Ethernet	`192.168.137.1`	Gateway, DHCP und NAT für den Server (ICS)
Ubuntu Server, enp6s0	`192.168.137.10/24`	statisch per Netplan
Ubuntu Server, tailscale0	`100.x.x.x`	DNS Server und Proxy für das Tailnet
Docker Netz proxy	`172.x.0.0/16`	gemeinsames Netz aller Weboberflächen

> Tailscale Adressen, MAC Adressen und Kontonamen sind in dieser Dokumentation anonymisiert.

3 Teil 1: Grundsystem

3.1 Ubuntu Server und Netplan

Der Server bekommt per Netplan eine feste Adresse im ICS Netz. Gateway und DNS zeigen auf den Windows PC.

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo netplan apply
ip route && ping -c 3 1.1.1.1
```

3.2 Docker und Portainer

Jeder Dienst läuft in einem eigenen Container. Portainer stellt eine Weboberfläche bereit, um Container und Stacks zu verwalten. Weil Portainer Zugriff auf den Docker Socket und damit volle Kontrolle über den Server hat, ist er nur an die LAN Adresse gebunden.

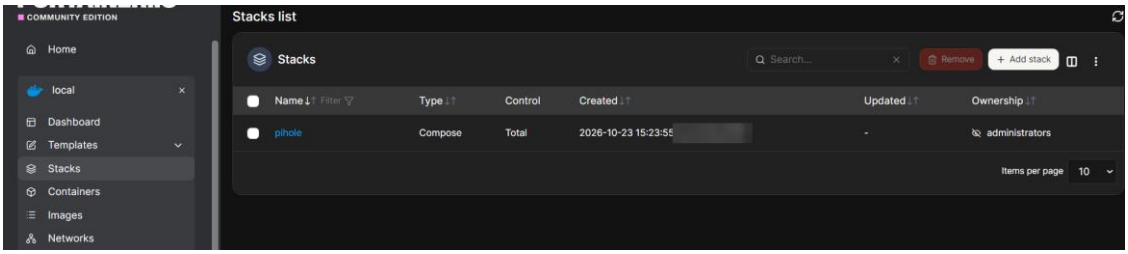


Abbildung 2: Stacks in Portainer

3.3 Pi-hole

Pi-hole ist der DNS Server des Netzes. Er übersetzt Namen in Adressen und beantwortet Anfragen an bekannte Werbe und Tracking Domains mit einer leeren Antwort. Port 53 ist gezielt nur auf der LAN und der Tailscale Adresse gebunden, damit der DNS Server nie aus fremden Netzen erreichbar ist.

3.4 Tailscale

Tailscale baut ein Mesh VPN auf WireGuard Basis auf und funktioniert auch hinter CGNAT, ganz ohne Portfreigabe. In der Admin Konsole ist der Server als globaler DNS Server eingetragen. Dadurch nutzen alle Handys im Tailnet den Pi-hole, auch unterwegs über mobile Daten.

3.5 SSH Härtung

Der Login erfolgt nur noch mit einem Ed25519 Schlüssel. Passwort Login und Root Login sind abgeschaltet. Der Schlüssel wurde erst übertragen und getestet, bevor der Passwort Login deaktiviert wurde, um sich nicht auszusperren.

```
sudo cp config/ssh/00-hardening.conf /etc/ssh/sshd_config.d/
sudo sshd -t && sudo systemctl restart ssh
```

4 Teil 2: Erweiterung

Vor der Umsetzung wurden alle Entscheidungen festgelegt und mit Begründung in `docs/ERWEITERUNG.md` dokumentiert. Die wichtigsten davon:

Thema	Entscheidung	Begründung
Struktur	Jeder Dienst ein eigener Stack unter `docker/<name>/` mit `.env.example`	Einzeln startbar und wartbar, gleiches Muster wie Teil 1
Netz	Ein gemeinsames Docker Netz `proxy` für alle Weboberflächen	Der Proxy erreicht Container über ihren Namen, ohne Host Ports
Ports	Nur der Proxy öffnet 443 (LAN und Tailscale), Verwaltung 81 nur im LAN	Eine einzige, kontrollierte Eingangstür
Namen	Wildcard `address=/home/<IP>` im Pi-hole	Jeder neue Dienst ist sofort unter `.home` erreichbar
HTTPS	Eigene CA mit mkcert, alle Namen als SAN	Let's Encrypt ist ohne öffentliche Domain und offene Ports nicht möglich
Dashboard	Homepage statt Homarr	Konfiguration als YAML im Repo statt Klicks in einer Datenbank
Socket	Homepage sieht Docker nur über einen Socket Proxy	Socket Zugriff ist Root Zugriff, es wird nur Lesen erlaubt
RAM	`mem_limit` auf allen neuen Containern	4 GB RAM gesamt, kein Dienst darf den Server lahmlegen

4.1 Was macht was

Komponente	Aufgabe
Nginx Proxy Manager	Die Eingangstür. Nimmt `https://grafana.home` entgegen und leitet zum richtigen Container weiter.
mkcert	Eigene kleine Zertifizierungsstelle, stellt das HTTPS Zertifikat für alle `.home` Namen aus.
Uptime Kuma	Der Wachhund. Prüft jede Minute alle Dienste und meldet Ausfälle über ntfy aufs Handy.
Nextcloud	Eigene Cloud wie Google Drive, für Dateien, Fotos und Kalender.
MariaDB, Redis, Cron	Datenbank, Zwischenspeicher und Hintergrund Jobs für Nextcloud.
Prometheus	Das Gedächtnis. Sammelt alle 30 Sekunden Messwerte und speichert sie 15 Tage.
Node Exporter	Liefert Werte des Servers: CPU, RAM, Platte, Netzwerk.
cAdvisor	Liefert den Verbrauch jedes einzelnen Containers.
Pi-hole Exporter	Liefert die Pi-hole Statistik: Anfragen, Blockquote, Clients.
Grafana	Der Bildschirm. Zeichnet aus den Prometheus Daten Live Diagramme.
Homepage	Startseite mit Kacheln und Live Werten aller Dienste.
docker-socket-proxy	Türsteher, über den Homepage nur lesen darf, welche Container laufen.

4.2 Reverse Proxy, Namen und HTTPS

So kommt eine Anfrage vom Handy bei Grafana an:

1. Das Handy fragt Pi-hole nach `grafana.home`. Die Wildcard Regel beantwortet jeden Namen unter `.home` mit der Adresse des Servers.
2. Der Browser baut HTTPS zum Nginx Proxy Manager auf. Das Zertifikat stammt von der eigenen CA, die auf den Geräten als vertrauenswürdig installiert ist.
3. Der Proxy leitet über das interne Docker Netz `proxy` an `grafana:3000` weiter. Grafana selbst hat keinen Port nach außen.

Adresse	Ziel im Docker Netz	Dienst
`https://dashboard.home`	`homepage:3000`	Homepage
`https://pihole.home`	`pihole:80`	Pi-hole
`https://npm.home`	`npm:81`	Nginx Proxy Manager
`https://kuma.home`	`uptime-kuma:3001`	Uptime Kuma
`https://grafana.home`	`grafana:3000`	Grafana
`https://cloud.home`	`nextcloud:80`	Nextcloud
`https://portainer.home`	`portainer:9443`	Portainer

Zertifikat, Proxy Hosts und alle Einstellungen wurden nicht von Hand geklickt, sondern über die REST API von Nginx Proxy Manager angelegt. Dadurch ist der Aufbau reproduzierbar.

4.3 Monitoring mit Prometheus und Grafana

Prometheus fragt alle 30 Sekunden vier Ziele ab: sich selbst, den Node Exporter (Server), cAdvisor (Container) und den Pi-hole Exporter. Grafana nutzt Prometheus als Datenquelle. Datenquelle und Dashboards werden beim Start automatisch per **Provisioning** eingerichtet:

- **Node Exporter Full** (ID 1860): CPU, RAM, Platte, Netzwerk und Temperatur
- **cAdvisor Exporter** (ID 14282): Verbrauch pro Container
- **Pi-hole Exporter** (ID 10176): Anfragen, geblockte Domains, Clients

Prometheus hat keinen Login und ist deshalb absichtlich nur intern erreichbar. Der Node Exporter läuft im Host Netzwerk, damit er die echten Netzwerkwerte sieht, lauscht aber nur auf der LAN Adresse.

4.4 Uptime Kuma und Alarme

Uptime Kuma prüft jede Minute zehn Ziele. Fällt eines zweimal hintereinander aus, kommt über **ntfy** eine Push Nachricht aufs Handy. Admin, Benachrichtigung, alle Monitore und die öffentliche Status Page legt das Skript `setup-kuma.js` in einem Schritt über die Socket.IO Schnittstelle an.

Monitor	Art	Ziel
Server Ping	Ping	`192.168.137.10`
Pi-hole DNS	DNS	Auflösung von `google.com` über den Pi-hole
Pi-hole Web, Portainer, NPM	HTTP	jeweilige Weboberfläche

Monitor	Art	Ziel
Nextcloud	HTTP mit Schlüsselwort	`status.php` muss `installed:true` melden
Grafana, Prometheus, Homepage	HTTP	Health Endpunkte
Internet	Ping	`1.1.1.1`

4.5 Nextcloud

Nextcloud läuft mit dem offiziellen Image, MariaDB 11.4 LTS als Datenbank, Redis für Cache und Dateisperren und einem eigenen Container für Hintergrund Jobs (Cron). Nur die App hängt am Netz `proxy`, Datenbank und Redis sind ausschließlich für Nextcloud erreichbar. Der Upload ist auf 10 GB pro Datei ausgelegt.

4.6 Homepage

Homepage ist die Startseite des Servers. Jede Kachel zeigt neben dem Link auch Live Werte, zum Beispiel geblockte Anfragen aus dem Pi-hole oder den Status aus Uptime Kuma. Die gesamte Konfiguration liegt als YAML im Repository. Passwörter werden über Umgebungsvariablen aus der `.env` eingesetzt.

4.7 Speicherbudget

Stack	Grenze	Typisch
Pi-hole, Portainer	keine	ca. 150 MB
Nginx Proxy Manager	256 MB	ca. 100 MB
Uptime Kuma	256 MB	ca. 170 MB
Nextcloud (App, DB, Redis, Cron)	768 + 512 + 128 + 128 MB	ca. 300 MB
Monitoring	512 + 384 + 64 + 64 + 192 MB	ca. 550 MB
Homepage und Socket Proxy	192 + 64 MB	ca. 130 MB

5 Sicherheit

Maßnahme	Wirkung
Keine Portfreigaben, Zugriff nur über Tailscale	Kein Angriffspunkt aus dem Internet, kein Missbrauch für DNS Amplification
Nur der Proxy öffnet einen Web Port (443)	Eine einzige, kontrollierte Eingangstür
Port 53 und 443 gezielt auf LAN und Tailscale Adresse gebunden	Nicht aus fremden Netzen erreichbar
Proxy Verwaltung (81) nur im LAN	Proxy Konfiguration nicht aus dem Tailnet änderbar
HTTPS mit eigener CA	Verschlüsselt auch im Tailnet, ohne öffentliche Domain
Socket Proxy mit reinem Lesezugriff für Homepage	Kein Root Zugriff über das Dashboard
Nextcloud Datenbank und Redis nicht im Netz `proxy`	Nur für Nextcloud selbst erreichbar
Eigenes Pi-hole App Passwort	Das Admin Passwort liegt in keiner Datei
`mem_limit` auf allen neuen Containern	Kein Dienst kann den RAM aufbrauchen
SSH nur mit Ed25519 Schlüssel	Schutz gegen Brute Force
Device approval im Tailnet	Neue Geräte brauchen manuelle Freigabe
Passwörter nur in `.env`, per `.gitignore` ausgeschlossen	Keine Zugangsdaten im Repository

6 Fehlersuche: Probleme und Lösungen

Jedes Problem wurde systematisch von unten nach oben eingegrenzt: Link, IP Adresse, Gateway, DNS, Dienst.

6.1 Grundsystem

Symptom	Ursache	Lösung
Server per Ping und SSH nicht erreichbar	Alte statische IP `192.168.31.27` aus der Installation, falsches Subnetz	Netplan neu konfiguriert
`netplan apply`: expected mapping	Zweite Datei `01-net.yaml` mit kaputter Einrückung	Datei entfernt, YAML neu geschrieben
Keine IP per DHCP	Bei der ICS war kein Zieladapter ausgewählt	Ethernet als Heimnetzwerkverbindung gewählt
Destination Host Unreachable	Gateway als `192.168.137.137` statt `.1` eingetragen	Mit `ip route` gefunden und korrigiert
Port 53 nicht bindbar	`systemd-resolved` belegt Port 53	`DNSStubListener=no`
Blocklisten laden nicht	Container ohne Namensauflösung beim Start	Fallback DNS im Container, Liste direkt von GitHub
Alle Tailscale Clients als `172.18.0.1`	Docker übersetzt die Quelladresse (NAT)	Verstanden und dokumentiert, Host Netzwerk als Option

6.2 Erweiterung

Symptom	Ursache	Lösung
Proxy startet nicht auf Port 80	CasaOS belegt `*:80` auf dem Host	Proxy nur auf 443, Adressen immer mit `https://`
Container heißen `cadviso` und `node-exporte`	Beim Übertragen von Windows wurde aus `\r` ein `r`, sed hat jedes r am Zeilenende gelöscht	Dateien neu kopiert, Skripte nur noch als Datei per scp übertragen
Grafana zeigt keine Dashboards	Provisioning Ordner nach scp nur für den Besitzer lesbar	Ordnerrechte auf 755 gesetzt
Serverlast 7,5, Grafana Fehler 502	cAdvisor scannt die Belegung jedes Containers, auf der HDD minutenlang	Disk Metriken abgeschaltet, Grafana mehr RAM
Uptime Kuma meldet 400 für Homepage	Homepage erlaubt nur bekannte Hostnamen	Internen Namen `homepage:3000` erlaubt
Pi-hole Passwort nicht auslesbar	Passwort in der Oberfläche gesetzt, nur als Hash gespeichert	Eigenes App Passwort über die Pi-hole API
PC hat plötzlich kein Internet	Tailscale am PC übernahm das DNS, war aber noch nicht freigegeben	`--accept-dns=false`, `.home` per hosts Datei
PC verbindet sich nicht mehr mit der FritzBox	Altes WPA3 Profil passte nicht zur geänderten WLAN Sicherheit	WLAN Profil gelöscht und neu verbunden

7 Ergebnis

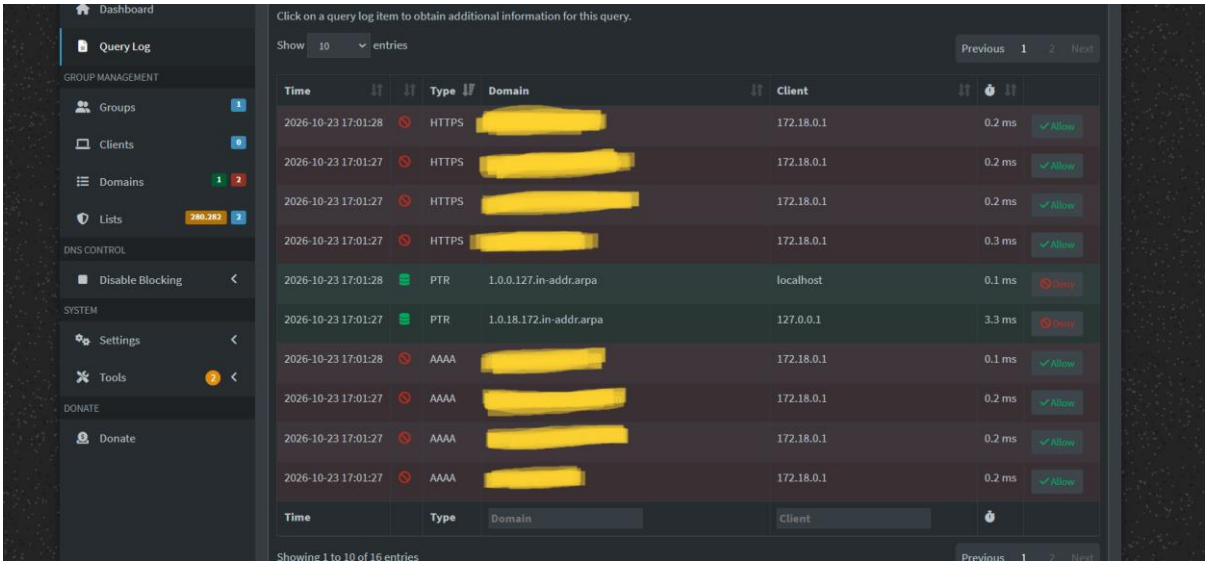
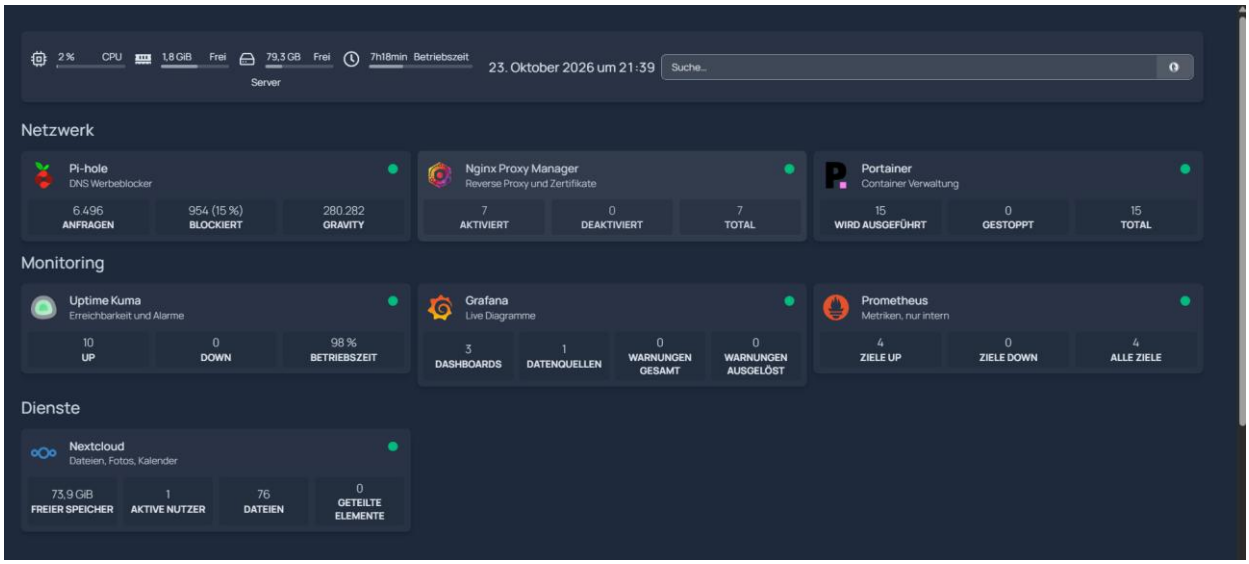


Abbildung 3: Geblockte Anfragen im Query Log des Pi-hole



Startseite (Homepage) mit Live Werten aller Dienste



Abbildung 5: Pi-hole Statistik in Grafana

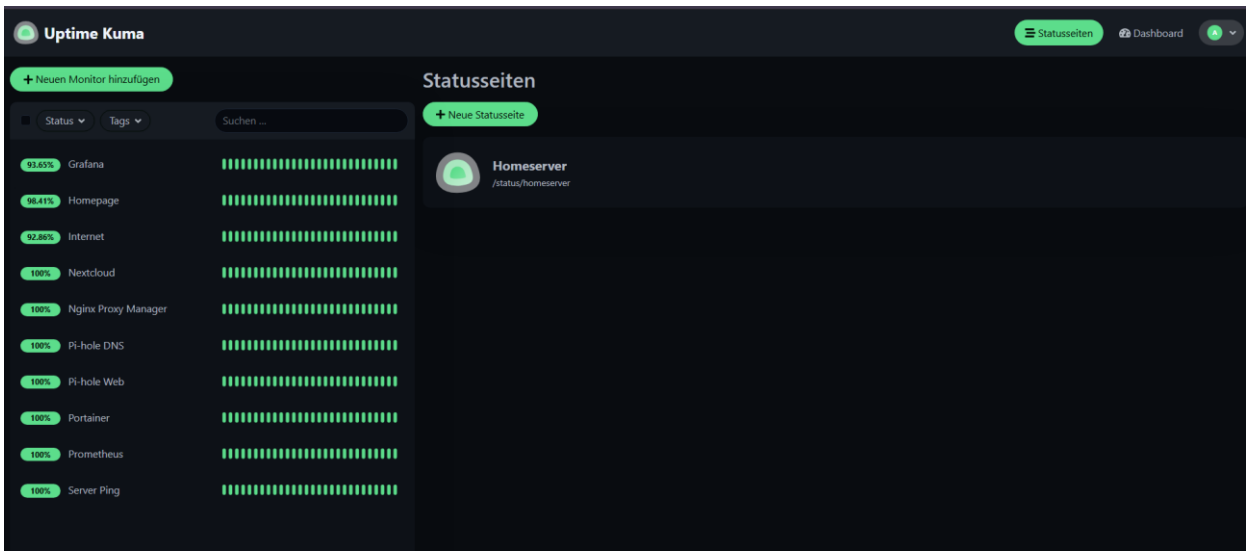


Abbildung 6: Uptime Kuma mit allen zehn Monitoren

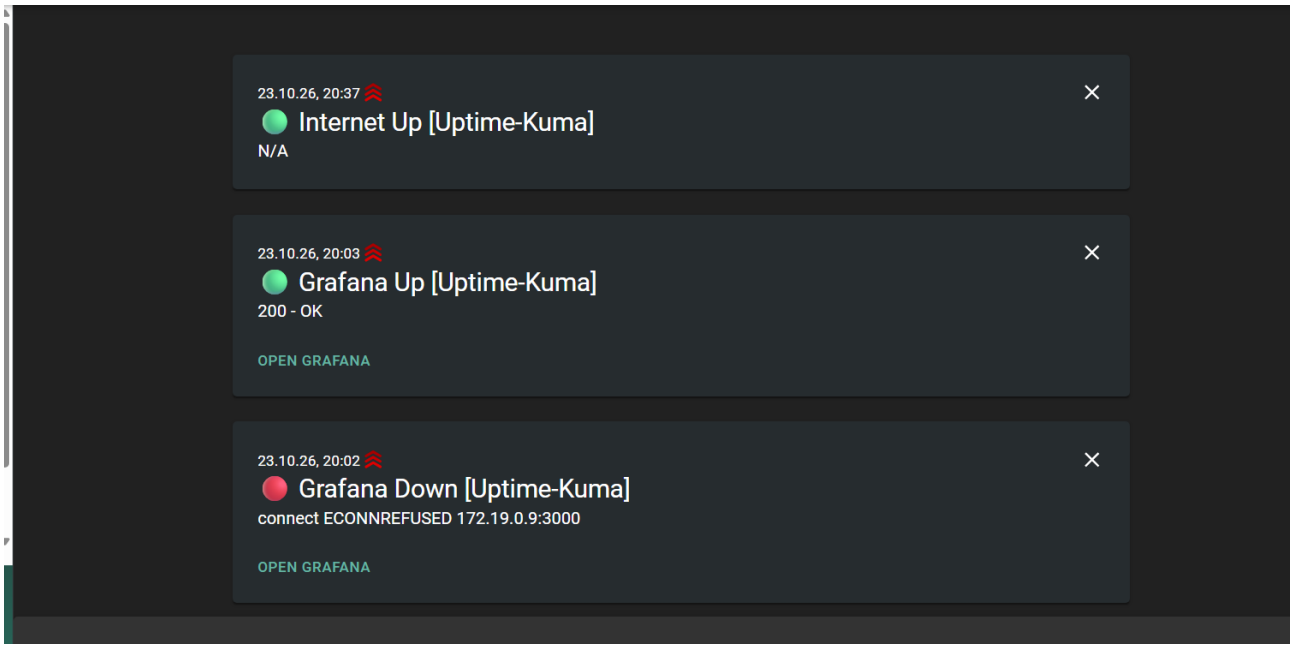


Abbildung 7: Ntfy Push-Benachrichtigungen mit UP/DOWN

Werbe und Tracking Anfragen vom Handy werden über das Tailnet zuverlässig geblockt. Alle Dienste sind über HTTPS mit sprechenden Namen erreichbar, werden minütlich überwacht und melden Ausfälle aufs Handy. Prometheus erreicht alle vier Ziele, Grafana zeigt die drei Dashboards, und alle zehn Monitore in Uptime Kuma sind grün.

Der Server ist komplett per SSH und Browser verwaltbar und von außen nicht angreifbar.

8 Was ich gelernt habe

- Wie die Windows Internetverbindungsfreigabe intern als kleiner Router mit DHCP und NAT arbeitet.
- Wie man Netzwerkfehler mit `ip a`, `ip route` und `ping` Schicht für Schicht eingrenzt.
- Wie Docker Bridge Netzwerke und Portbindungen funktionieren und warum dadurch Quelladressen verloren gehen.
- Wie ein Mesh VPN ohne Portfreigaben durch CGNAT kommt.
- Wie ein Reverse Proxy mehrere Dienste hinter einer Adresse bündelt und wie Zertifikate und eine eigene CA zusammenspielen.
- Wie Prometheus Metriken einsammelt, Grafana sie darstellt und warum Monitoring auf schwacher Hardware selbst zur Last werden kann.
- Dass ein VPN, das das DNS übernimmt, einem PC komplett das Internet nehmen kann, und wie man das sauber trennt.